

UNIVERSIDADE FEDERAL DE VIÇOSA

CURSO DE ESPECIALIZAÇÃO LATO SENSU EM ROBÓTICA

QUARTA TURMA

DATA: 08ABR2026

ALUNO: FABIO PINHEIRO CARDOSO

1 - Escopo do Projeto

1.1 - Introduction

<https://community.wolfram.com/groups/-/m/t/1502340>

A odometria é uma técnica amplamente utilizada para estimar, de forma aproximada, a posição e a orientação de um robô móvel ao longo do tempo, com base na medição do deslocamento de suas rodas.

Um dos sensores mais comuns usados para esse fim é o *encoder* incremental, um dispositivo que conta os giros (ou incrementos de rotação) de cada roda, permitindo calcular a distância percorrida.

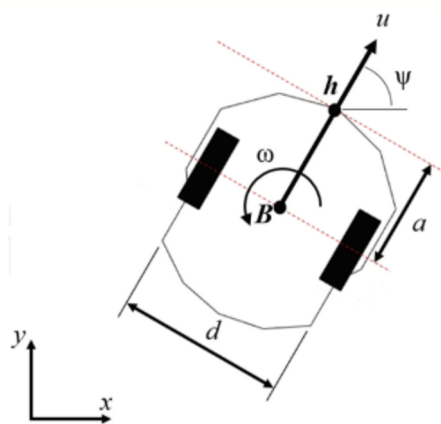
A ideia por trás do processo de odometria envolve a integração numérica do movimento do robô ao longo do tempo. É evidente que qualquer erro proveniente dos sensores — como imprecisões na leitura da direção ou distância percorrida — interfere diretamente na determinação da posição. Isso ocorre porque esses erros tendem a se acumular ao longo do tempo, à medida que o robô percorre o caminho em busca de seu destino.

Esse método é bastante viável quando o destino está relativamente próximo do ponto de partida, pois é simples de implementar e fornece uma boa aproximação da posição.

A odometria é normalmente realizada com base nos dados dos *encoders* localizados nas rodas do robô, assumindo que as revoluções das rodas correspondem a deslocamentos lineares em relação ao solo.

Considere um robô diferencial com duas rodas, cada uma equipada com um *encoder* incremental. A distância entre as rodas (eixo) " d " é de 0,4 m, e o raio de cada roda (direita rR e esquerda rL) é 0,1 m, ou seja, $r = rR = rL$.

Assuma que o robô parte da posição inicial $(x_0, y_0) = (0, 0)$ com orientação $\psi_0 = 0 \text{ rad}$.



Para robôs móveis com tração diferencial, como o robô uniciclo da figura, as velocidades das rodas podem ser usadas para calcular as velocidades linear e angular do robô em relação ao solo. Que são sinais de controle de

alto nível para o robô. Dadas a velocidade angular da roda direita “ ω_R ” e da roda esquerda “ ω_L ”, expressa em rad/s , as equações para determinar a velocidade do robô são:

- Velocidade linear: $u = r \frac{\omega_R + \omega_L}{2}$
- Velocidade angular: $\omega = r \frac{\omega_R - \omega_L}{d}$

Com essas equações, é possível estimar o movimento do robô a partir dos dados de velocidade medidos por encoders instalados nas rodas.

O modelo cinemático de um robô do tipo unicycle com deslocamento do ponto de controle “ h ” é dado por:

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\psi} \end{bmatrix} = \begin{bmatrix} \cos \psi & -a \sin \psi \\ \sin \psi & a \cos \psi \\ 0 & 1 \end{bmatrix} \begin{bmatrix} u \\ \omega \end{bmatrix}$$

Onde:

“(x, y)” representa a posição do robô no plano cartesiano.

“ ψ ” é o ângulo de orientação do robô em relação ao “eixo x ”.

“ u ” é a velocidade linear e “ ω ” é a velocidade angular aplicada ao robô.

“ a ” é a distância entre o eixo virtual que une as rodas do robô e o ponto de aplicação do controle.

Em função dos dados de velocidade das rodas direita e esquerda do robô, determine a rota navegada, sabendo que “ a ” = 0,10 m.

Para tal, realize os seguintes passos:

1. Realize a leitura do arquivo de dados “ $\omega_R \omega_L.txt$ ”, o qual possui 400 linhas e 3 colunas. Em outras palavras, o arquivo contém 400 medidas de tempo, velocidade da roda direita “ ω_R ” e velocidade da roda esquerda “ ω_L ”.
2. Para demonstrar a evolução temporal das velocidades das rodas, plote os gráficos que relaciona o “tempo x ω_R ” e o “tempo x ω_L ”.
3. Transforme as velocidades “ ω_R ” e “ ω_L ” em sinais de controle “ u ” e “ ω ”.
4. Para demonstrar a evolução temporal dos sinais de controle do robô, plote os gráficos que relacionam o “tempo x u ” e o “tempo x ω ”.
5. Use o modelo cinemático, para determinar a posição e orientação do robô ao longo do tempo. A título de exemplo, para determinar a orientação do robô, faça:

$$\dot{\psi}(k) = [\psi(k) - \psi(k-1)] / \Delta t = \omega,$$

ou ainda:

$$\psi(k) = \psi(k-1) + \omega \Delta t.$$

6. Para demonstrar a evolução temporal da posição e orientação do robô, plote os gráficos que relaciona o “tempo x x ” e o “tempo x y ”, o “tempo x ψ ”.
7. Para demonstrar a navegação do robô no plano cartesiano, plote o gráfico que relaciona x e y .
8. Informe numericamente a posição final do robô ao final do experimento.

1.2 - Functions to aquisition, graphics and processing the dataset

1.2.1 - Functions to aquisition the dataset

```
In[2]:= ImportingFile[tagOS_(*"W"_indows or "L"_inux*), DataType_(*"DataType": "Table",
                                         |>|labela
                                         "Data", etc*), directory1_(*"dir"*), directory2_(*"dir"*), filename_(*"filename"*), extensionfile_] :=
Import[FileNameJoin[{If[TemplateApply[""], directory1] # TemplateApply["", directory2], ParentDirectory[
|>|importa |>|une nome do arq... |>| aplica modelo padrão |>| aplica modelo padrão |>| diretório predecessor
ParentDirectory[Evaluate[NotebookDirectory[]]], NotebookDirectory[], If[TemplateApply["", tagOS] = "W",
|>| diretório predecessor |>| calcula |>| diretório do notebook |>| diretório do notebook |>| aplica modelo padrão
TemplateApply[""/"/".", {directory1, directory2, filename, extensionfile}], If[TemplateApply["", tagOS] = "L",
|>| aplica modelo padrão |>| aplica modelo padrão
TemplateApply[""/"/"/".", {directory1, directory2, filename, extensionfile}]]]], DataType]
```

1.2.2 - Graphics

```

In[3]:= graphs1[data_, name_] :=
ListPlot[data, Joined → True, GridLines → Automatic, ImageSize → Large, PlotRange → All, PlotLabel →
|gráfico de uma l...|unido|ver...|grade de linhas|automático|tamanho da ...|grande|intervalo do g...|l...|etiqueta de gráfico
Style[If[Length[name] > 1, TemplateApply["t vs `` curves", StringRiffle[name, ","], TemplateApply["t vs `` curve", name]],
|estilo|l...|comprimento|aplica modelo padrão|intercala uma cadeia de cara...|aplica modelo padrão
FontSize → 24(*Large*)], AxesStyle → Directive[FontFamily → "Times", 20],
|tamanho da fonte|grande|estilo dos eixos|diretiva|família da fonte|multiplicação
LabelStyle → Directive[Black, Bold], AxesLabel → {Style["time (s)", 20, Bold, Black],
|estilo de etiqu...|diretiva|preto|negrito|legenda dos ei...|estilo|negr...|preto
Style[If[Length[name] > 1, StringRiffle[Table[TemplateApply["", name[n]], {n, 1, Length[name]}], "", ],
|estilo|l...|comprimento|intercala um...|tabela|aplica modelo padrão|comprimento
TemplateApply["", name]], 20, Bold, Black]], LabelStyle → Directive[Black, Bold],
|aplica modelo padrão|negr...|preto|estilo de etiqu...|diretiva|preto|negrito
PlotLegends → Placed[If[Length[name] > 1, Table[TemplateApply["", name[n]], {n, 1, Length[name]}],
|legenda do gráfico|situação|l...|comprimento|tabela|aplica modelo padrão|comprimento
TemplateApply["", name]], Below], Background → LightGray]
|aplica modelo padrão|abaixo|imagem de fundo|cinza claro

In[4]:= graphs2[data_, name_, unit_] :=
ListPlot[data, Joined → True, GridLines → Automatic, ImageSize → Large, PlotRange → All, PlotLabel →
|gráfico de uma l...|unido|ver...|grade de linhas|automático|tamanho da ...|grande|intervalo do g...|l...|etiqueta de gráfico
Style[If[Length[name] > 1, TemplateApply["t vs `` curves", StringRiffle[name, ","], TemplateApply["t vs `` curve", name]],
|estilo|l...|comprimento|aplica modelo padrão|intercala uma cadeia de cara...|aplica modelo padrão
FontSize → 24(*Large*)], AxesStyle → Directive[FontFamily → "Times", 20],
|tamanho da fonte|grande|estilo dos eixos|diretiva|família da fonte|multiplicação
LabelStyle → Directive[Black, Bold], AxesLabel → {Style["time (s)", 20, Bold, Black],
|estilo de etiqu...|diretiva|preto|negrito|legenda dos ei...|estilo|negr...|preto
Style[If[Length[name] > 1, StringRiffle[Table[TemplateApply["" `"), {name[n], unit[n]], {n, 1, Length[name]}], "", ],
|estilo|l...|comprimento|intercala um...|tabela|aplica modelo padrão|comprimento
TemplateApply["" `"), {name, unit}]], 20, Bold, Black]], LabelStyle → Directive[Black, Bold],
|aplica modelo padrão|negr...|preto|estilo de etiqu...|diretiva|preto|negrito
PlotLegends → Placed[If[Length[name] > 1, Table[TemplateApply["" `"), {name[n], unit[n]], {n, 1, Length[name]}],
|legenda do gráfico|situação|l...|comprimento|tabela|aplica modelo padrão|comprimento
TemplateApply["" `"), {name, unit}]], Below], Background → LightGray]
|aplica modelo padrão|abaixo|imagem de fundo|cinza claro

In[310]:=
graphs3[data_, title_, AxesNames_, units_] :=
ListPlot[data, Joined → True, GridLines → Automatic, ImageSize → Large, PlotRange → All, PlotLabel →
|gráfico de uma l...|unido|ver...|grade de linhas|automático|tamanho da ...|grande|intervalo do g...|l...|etiqueta de gráfico
Style[If[Length[title] > 1, TemplateApply["", StringRiffle[title, " - "], title], FontSize → 24, FontFamily → "Times"],
|estilo|l...|comprimento|aplica modelo padrão|intercala uma cadeia de caracteres|tamanho da fonte|família da fonte|multiplicação
AxesStyle → Directive[FontFamily → "Times", 20], LabelStyle → Directive[Black, Bold],
|estilo dos eixos|diretiva|família da fonte|multiplicação|estilo de etiqu...|diretiva|preto|negrito
AxesLabel → {Style[TemplateApply["" `"), {AxesNames[1], units[1]}], 20, Bold, Black],
|legenda dos ei...|estilo|aplica modelo padrão|negr...|preto
Style[TemplateApply["" `"), {AxesNames[2], units[2]}], 20, Bold, Black]], LabelStyle → Directive[Black, Bold],
|estilo|aplica modelo padrão|negr...|preto|estilo de etiqu...|diretiva|preto|negrito
PlotLegends → Placed[If[Length[title] > 1, Table[TemplateApply["" `"), {title[n], units[n]], {n, 1, Length[title]}],
|legenda do gráfico|situação|l...|comprimento|tabela|aplica modelo padrão|comprimento
TemplateApply["" `"), {title, units[1]}]], Below], Background → LightGray, AspectRatio → 4.55/3]
|aplica modelo padrão|abaixo|imagem de fundo|cinza claro|quociente de aspecto

In[309]:=
graphs4[data_, title_, LegendsName_, AxesNames_, units_] :=
ListPlot[data, Joined → True, GridLines → Automatic, ImageSize → Large, PlotRange → All, PlotLabel →
|gráfico de uma l...|unido|ver...|grade de linhas|automático|tamanho da ...|grande|intervalo do g...|l...|etiqueta de gráfico
Style[If[Length[title] > 1, TemplateApply["", StringRiffle[title, " - "], title], FontSize → 24, FontFamily → "Times"],
|estilo|l...|comprimento|aplica modelo padrão|intercala uma cadeia de caracteres|tamanho da fonte|família da fonte|multiplicação
AxesStyle → Directive[FontFamily → "Times", 20], LabelStyle → Directive[Black, Bold],
|estilo dos eixos|diretiva|família da fonte|multiplicação|estilo de etiqu...|diretiva|preto|negrito
AxesLabel → {Style[TemplateApply["" `"), {AxesNames[1], units[1]}], 20, Bold, Black],
|legenda dos ei...|estilo|aplica modelo padrão|negr...|preto
Style[TemplateApply["" `"), {AxesNames[2], units[2]}], 20, Bold, Black]],
|estilo|aplica modelo padrão|negr...|preto
LabelStyle → Directive[Black, Bold], PlotLegends →
|estilo de etiqu...|diretiva|preto|negrito|legenda do gráfico
Placed[If[Length[LegendsName] > 1, Table[TemplateApply["", LegendsName[n]], {n, 1, Length[LegendsName]}],
|situação|l...|comprimento|tabela|aplica modelo padrão|comprimento
TemplateApply["", LegendsName]], Below], Background → LightGray, AspectRatio → 4.55/3]
|aplica modelo padrão|abaixo|imagem de fundo|cinza claro|quociente de aspecto

```

```

In[319]:=
graphs5[data_, title_, LegendsName_, AxesNames_, units_] :=
  ListPlot[data, Joined → True, GridLines → Automatic, ImageSize → Large, PlotRange → All, PlotLabel →
    gráfico de uma l... |unido |ver... |grade de linhas |automático |tamanho da ... |grande |intervalo do g... |t... |etiqueta de gráfico
    Style[If[Length[title] > 1, TemplateApply["", StringRiffle[title, " - "]], title], FontSize → 24, FontFamily → "Times"],
    estilo l... |comprimento |aplica modelo padrão |intercala uma cadeia de caracteres |tamanho da fonte |família da fonte |multiplicaçã
    AxesStyle → Directive[FontFamily → "Times", 20], LabelStyle → Directive[Black, Bold],
    estilo dos eixos |diretiva |família da fonte |multiplicação |estilo de etiqu... |diretiva |preto |negrito
    AxesLabel → (Style[TemplateApply["" (`)", {AxesNames[1], units[1]}], 20, Bold, Black],
    legenda dos eix... |estilo |aplica modelo padrão |negr... |preto
    Style[TemplateApply["" (`)", {AxesNames[2], units[2]}], 20, Bold, Black]),
    estilo |aplica modelo padrão |negr... |preto
    LabelStyle → Directive[Black, Bold], PlotLegends →
    estilo de etiqu... |diretiva |preto |negrito |legenda do gráfico
    Placed[If[Length[LegendsName] > 1, Table[TemplateApply["", LegendsName[n]], {n, 1, Length[LegendsName]}],
    situado l... |comprimento |etiqueta |aplica modelo padrão |comprimento
    TemplateApply["", LegendsName]], Below], Background → LightGray, AspectRatio → 1/2]
    aplica modelo padrão |abaixo |imagem de fundo |cinza claro |quociente de aspecto

```

1.2.3 - Auxiliary Formulas

1.2.3.a - Rectangular Integral Methods - Right

i - Rectangular Integral Formulas - Right - Starting in the first element

```

In[108]:=
RightRectIntV3[dataset_, interval_(*"C"onstant "I"ntervals or "V"ariables "I"ntervals*)] :=
  símbolo de ... |unidade imaginária |unidade imaginária

If[Length[Dimensions[dataset]] == 1 && interval == "CI",
  l... |compr... |dimensões
  
$$\left( \frac{\text{dataset}[-1] - \text{dataset}[1]}{\text{Length}[\text{dataset}]} \right) \text{Sum}[\text{dataset}[i + 1] (*\text{The interval is regular but it is not specified*}), \{i, 1, \text{Length}[\text{dataset}] - 1\}],$$

  |soma |comprimento

If[Length[Dimensions[dataset]] == 2 && interval == "CI",
  l... |compr... |dimensões
  
$$\left( \frac{\text{dataset}[-1, 1] - \text{dataset}[1, 1]}{\text{Dimensions}[\text{dataset}][1]} \right) \text{Sum}[\text{dataset}[i + 1, 2] (*\text{The interval is a constant*}), \{i, 1, \text{Dimensions}[\text{dataset}][1] - 1\}],$$

  |soma |dimensões

If[Length[Dimensions[dataset]] == 2 && interval == "VI", Sum[(dataset[i + 1, 2]) (dataset[i + 1, 1] - dataset[i, 1])
  l... |compr... |dimensões |soma
  (* If the interval is variable*), {i, 1, Dimensions[dataset][1] - 1}]]]
  |se |dimensões

(* Rectangular Integration Formula to initial values starts in the first value. *)

```

ii - Rectangular Integral Formulas - Right - Starting in the desired element

```

In[7]:=
RightRectIntV4[dataset_, interval_(*"C"onstant "I"ntervals or "V"ariables "I"ntervals*), initialpoint_, finalpoint_] :=
  símbolo de ... |unidade imaginária |unidade imaginária

If[Length[Dimensions[dataset]] == 1 && interval == "CI",  $\left( \frac{\text{dataset}[\text{finalpoint}] - \text{dataset}[\text{initialpoint}]}{\text{Length}[\text{dataset}[\text{initialpoint} ;; \text{finalpoint}]]} \right)$ 
  l... |compr... |dimensões

  Sum[dataset[i + 1] (*If the interval is not specified*), {i, initialpoint, finalpoint - 1}],
  |soma |se

If[Length[Dimensions[dataset]] == 2 && interval == "CI",  $\left( \frac{\text{dataset}[\text{finalpoint}, 1] - \text{dataset}[\text{initialpoint} - 1, 1]}{\text{Dimensions}[\text{dataset}[\text{initialpoint} ;; \text{finalpoint}][1]} \right)$ 
  l... |compr... |dimensões

  Sum[dataset[i + 1, 2] (*If the interval is constant*), {i, initialpoint, finalpoint - 1}],
  |soma |se

If[Length[Dimensions[dataset]] == 2 && interval == "VI", Sum[(dataset[i + 1, 2]) (dataset[i, 1] - dataset[i - 1, 1])
  l... |compr... |dimensões |soma
  (* If the interval is variable*), {i, initialpoint, finalpoint - 1}]]] (* Rectangular Integration Formula. *)
  |se

```

iii - Integrate Function - Right Rectangular Method

```

In[8]:=
(*integrateFunctionBackRect[dataset_, interval_(*"C"onstant "I"ntervals or "V"ariables "I"ntervals*)] :=Block[[],
  símbolo de ... |unidade imaginária |unidade imaginária |bloqueia

  Return[]]*)
  |retorna

```

```

In[9]:= (*Table[Piecewise[{{BackRectIntV3[dataset[[1;;i]],interval],1<=Length[dataset]-1},
      |etiqueta |função por partes |comprimento
      {BackRectIntV4[dataset,interval,Length[dataset]-2,Length[dataset]-1,i=Length[dataset]]},i,Length[dataset]]]*)

In[* ]:= (*integrateFunctionRightRect[dataset_, interval_(*"C"onstant "I"ntervals or "V"ariables "I"ntervals*) :=
      |símbolo de ... |unidade imaginária |unidade imaginária
      Table[Piecewise[{{RightRectIntV3[dataset[[1;;i]],interval],1<=Length[dataset]-1},
      |etiqueta |função por partes |comprimento
      {RightRectIntV4[dataset,interval,Length[dataset]-1,Length[dataset]],i=Length[dataset]]},i,Length[dataset]]]*)

In[* ]:= (*integrateFunctionRightRect[dataset_, interval_(*"C"onstant "I"ntervals or "V"ariables "I"ntervals*) :=
      |símbolo de ... |unidade imaginária |unidade imaginária
      Table[RightRectIntV3[dataset[[1;;i]],interval],i,Dimensions[dataset][[1]])*)

In[112]:=
integrateFunctionRightRect[dataset_, initialValues_, intervalType_] :=
  Block[{t0 = If[Length[Dimensions[dataset]] == 2, initialValues[[1]], f0 = If[Length[Dimensions[dataset]] == 2, initialValues[[2]],
    |bloqueia |comprimento |dimensões |comprimento |dimensões
    solist = If[Length[Dimensions[dataset]] == 2, {{initialValues[[1]], initialValues[[2]]}, {initialValues[[1]]},
    |comprimento |dimensões
    tf = If[Length[Dimensions[dataset]] == 2, dataset[[All, 1]] - 1]],
    |comprimento |dimensões |tudo
    ff = If[Length[Dimensions[dataset]] == 2, dataset[[All, 2]] - 1], dataset[-1]],
    |comprimento |dimensões |tudo
    dts = dataset, n = 2, tn, fn, told, fold, steps, h, tnew, fnew),
    steps = Length[dts[[All, 1]]] - 1;
    |comprimento |tudo
    (*told=t0;
    fold=f0;*)
    Do[
    |repete
      tnew = dts[[All, 1]][[n]];
      |tudo
      fnew = RightRectIntV3[dts[[1 ;; n]], intervalType];
      solist = Append[solist, {tnew, fnew}]; (*Mounting the result vector*)
      |adiciona
      n = n + 1, {steps}];
    Return[solist] (*to see all grid points*)
    |retorna

```

1.2.3.b - Rectangular Integral Methods - Center Point Method

i - Center Point Method Formula - Starting in the first element

```

(*CenterRectIntV5[dataset_,interval_(*"C"onstant "I"ntervals or "V"ariables "I"ntervals*)]:=
      |símbolo de ... |unidade imaginária |unidade imaginária
      If[Length[Dimensions[dataset]]==1&&interval=="CI",1/2) Sum[(dataset[[i+1]]+dataset[[i]]),(i,1,Length[dataset]-1)],
      |comprimento |dimensões |soma |comprimento
      If[Length[Dimensions[dataset]]==2&&interval=="CI",((dataset[[2,1]]-dataset[[1,1]]
      |comprimento |dimensões
      Sum[(dataset[[i+1,2]]+dataset[[i,2]]),(i,1,Length[dataset]-1)),If[Length[Dimensions[dataset]]==2&&interval=="VI",
      |soma |comprimento |comprimento |dimensões
      Sum[(dataset[[i+1,1]]-dataset[[i,1]])/2)(dataset[[i+1,2]]-dataset[[i,2]]),(i,1,Dimensions[dataset][[1]]-1)]]]]
      |soma |dimensões
      (* Center Point Method Formula to initial values starts in the first value. *)*)
      |centro |ponto |método

```

ii - Center Point Method Formula - Starting in the desired element

```

In[*]:= (*CenterRectIntV6[dataset_,interval_(*"C"onstant "I"ntervals or "V"ariables "I"ntervals*),initialpoint_,finalpoint_]:=
      |símbolo de ... |unidade imaginária |unidade imaginária
      If[Length[Dimensions[dataset]]==1&&interval=="CI",
      |... |compr... |dimensões
      (1/2) Sum[(dataset[[i+1]]+dataset[[i]]),(i,initialpoint,finalpoint)],If[Length[Dimensions[dataset]]==2&&interval=="CI",
      |soma |... |compr... |dimensões
      ( (dataset[[2,1]]-dataset[[1,1]]) Sum[(dataset[[i+1]]+dataset[[i]]),(i,initialpoint,finalpoint)],
      |soma
      If[Length[Dimensions[dataset]]==2&&interval=="VI",Sum[( (dataset[[i+1,1]]-dataset[[i,1]]) (dataset[[i+1,2]]-dataset[[i,2]])+
      |... |compr... |dimensões |soma
      (dataset[[i,2]](dataset[[i+1,1]]-dataset[[i,1]])),(i,initialpoint,finalpoint)]]]]
      (* Center Point Method Formula to initial values starts in the first value. *)*)
      |centro |ponto |método

```

iii - Integrate Function - Center Point Method

```

In[*]:= (*integrateFunctionCentPtRect[dataset_, interval_(*"C"onstant "I"ntervals or "V"ariables "I"ntervals*) :=
      |símbolo de ... |unidade imaginária |unidade imaginária
      Table[Piecewise[{{CenterRectIntV5[dataset[[1;;i]],interval],1<=i<=Length[dataset]-1},
      |tabela |função por partes |comprimento
      {CenterRectIntV6[dataset,interval,Length[dataset]-2,Length[dataset]-1],i==Length[dataset]}]],(i,Length[dataset])
      |comprimento |comprimento |comprimento |comprimento
      (* (Length[dataset]-2, Length[dataset]-1) adopted for correct the "integrateFunctionv2" *)*)
      |comprimento |comprimento

```

1.2.3.c - Rectangular Integral Methods - Left**i - Rectangular Integral Formulas - Left - Starting in the first element**

```

In[113]:= LeftRectIntV7[dataset_, interval_(*"C"onstant "I"ntervals or "V"ariables "I"ntervals*) :=
      |símbolo de ... |unidade imaginária |unidade imaginária
      If[Length[Dimensions[dataset]] == 1 && interval == "CI",
      |... |compr... |dimensões
      ( (dataset[[-1]] - dataset[[1]]) Sum[dataset[[i]](*The interval is regular but it is not specified*), {i, 1, Length[dataset] - 1}],
      |soma |comprimento
      If[Length[Dimensions[dataset]] == 2 && interval == "CI",
      |... |compr... |dimensões
      ( (dataset[[-1, 1]] - dataset[[1, 1]]) Sum[dataset[[i, 2]](*The interval is a constant*), {i, 1, Dimensions[dataset][[1] - 1}],
      |soma |dimensões
      If[Length[Dimensions[dataset]] == 2 && interval == "VI",
      |... |compr... |dimensões
      Sum[(dataset[[i, 2]] (dataset[[i+1, 1]] - dataset[[i, 1]])(* If the interval is variable*), {i, 1, Dimensions[dataset][[1] - 1}]]]]
      |soma |se |dimensões
      (* Rectangular Integration Formula to initial values starts in the first value. *)

```

ii - Rectangular Integral Formulas - Left - Starting in the desired element

```

In[15]:= LeftRectIntV8[dataset_, interval_(*"C"onstant "I"ntervals or "V"ariables "I"ntervals*), initialpoint_, finalpoint_] :=
    |símbolo de ... |unidade imaginária |unidade imaginária
    If[Length[Dimensions[dataset]] == 1 && interval == "CI",
    |... |compr... |dimensões
    (
        dataset[finalpoint] - dataset[1]
    )
    |soma |se
    Sum[dataset[i](*If the interval is not specified*), {i, initialpoint, finalpoint}],

    If[Length[Dimensions[dataset]] == 2 && interval == "CI", (
        dataset[finalpoint, 1] - dataset[initialpoint, 1]
    )
    |... |compr... |dimensões
    (
        Sum[dataset[i, 2](*If the interval is constant*), {i, initialpoint, finalpoint}], If[Length[Dimensions[dataset]] == 2 &&
        |soma |se
        interval == "VI", Sum[(dataset[initialpoint, 2]) (dataset[finalpoint, 1] - dataset[initialpoint, 1])
        |soma
        (*If the interval is variable*), {i, initialpoint, finalpoint}]]] (* Rectangular Integration Formula. *)
    |se

```

iii - Integrate Function - Left Rectangular Method

```

(*integrateFunctionLeftRect[dataset_, interval_(*"C"onstant "I"ntervals or "V"ariables "I"ntervals*)] :=
    |símbolo de ... |unidade imaginária |unidade imaginária
    Table[LeftRectIntV7[dataset[[1;;i], interval], {i, Length[dataset]}] *)
    |etiqueta |comprimento

In[114]:=
integrateFunctionLeftRect[dataset_, initialValues_, intervalType_] :=
    Block[{t0 = If[Length[Dimensions[dataset]] == 2, initialValues[1]], f0 = If[Length[Dimensions[dataset]] == 2, initialValues[2]],
    |bloqueia |... |compr... |dimensões |... |compr... |dimensões
    sollist = If[Length[Dimensions[dataset]] == 2, {{initialValues[1], initialValues[2]}, {initialValues[1]}},
    |... |compr... |dimensões
    tf = If[Length[Dimensions[dataset]] == 2, dataset[All, 1][[1]],
    |... |compr... |dimensões |tudo
    ff = If[Length[Dimensions[dataset]] == 2, dataset[All, 2][[1]], dataset[-1]],
    |... |compr... |dimensões |tudo
    dts = dataset, n = 2, tn, fn, told, fold, steps, h, tnew, fnew},
    steps = Length[dts[All, 1]] - 1;
    |comprimento |tudo
    (*told=t0;
    fold=f0;*)
    Do[
    |repete
    tnew = dts[All, 1][[n]];
    |tudo
    fnew = LeftRectIntV7[dts[[1 ;; n], intervalType];
    sollist = Append[sollist, {tnew, fnew}]; (*Mounting the result vector*)
    |adiciona
    n = n + 1, {steps}];
    Return[sollist] (*to see all grid points*)
    |retorna

In[* ]:= (*integrateFunctionLeftRect[dataset_, interval_(*"C"onstant "I"ntervals or "V"ariables "I"ntervals*)] :=
    |símbolo de ... |unidade imaginária |unidade imaginária
    Table[Piecewise[{{LeftRectIntV8[dataset, interval, 1, 2], i == 1}, {LeftRectIntV7[dataset[[1;;i], interval], 2 <= i <= Length[dataset]}]},
    |etiqueta |função por partes |comprimento
    {i, Length[dataset]}] *)
    |comprimento

```

1.2.3.d - Trapezoidal Integral Methods

i - Trapezoidal Integral Formulas - Starting in the first element

```

In[85]:= TrapIntV11[dataset_, interval_(*"C"onstant "I"ntervals or "V"ariable "I"nterval*)] :=

$$\text{If}\left[\frac{\text{dataset}[-1] - \text{dataset}[1]}{2 (\text{Length}[\text{dataset}] - 1)} \text{Sum}[(\text{dataset}[i] + \text{dataset}[i + 1])(\text{If the interval is not specified}), (i, 1, \text{Length}[\text{dataset}] - 1)],\right.$$



$$\text{If}\left[\frac{\text{dataset}[-1, 1] - \text{dataset}[1, 1]}{2 (\text{Dimensions}[\text{dataset}][[1] - 1)} \text{Sum}[(\text{dataset}[i, 2] + \text{dataset}[i + 1, 2])(\text{If the interval is constant}), (i, 1, \text{Dimensions}[\text{dataset}][[1] - 1)],\right.$$



$$\text{If}[\text{Length}[\text{Dimensions}[\text{dataset}]] = 2 \ \&\& \ \text{interval} = \text{"VI"}, (1/2) \text{Sum}[(\text{dataset}[i, 2] + \text{dataset}[i + 1, 2])(\text{dataset}[i + 1, 1] - \text{dataset}[i, 1])(\text{If the interval is variable}), (i, 1, \text{Dimensions}[\text{dataset}][[1] - 1)]]](\text{"Trapezoidal Integration Formula. *})$$


```

ii - Trapezoidal Integral Formulas - Starting in the desired element

```

In[18]:= TrapIntV12[dataset_, interval_(*"C"onstant "I"ntervals or "V"ariables "I"ntervals*), initialpoint_, finalpoint_] :=

$$\text{If}\left[\frac{\text{dataset}[-1] - \text{dataset}[1]}{2 \text{Length}[\text{dataset}]} \text{Sum}[(\text{dataset}[i] + \text{dataset}[i + 1])(\text{If the interval is not specified}), (i, \text{initialpoint}, \text{finalpoint})],\right.$$



$$\text{If}\left[\frac{\text{dataset}[-1, 1] - \text{dataset}[1, 1]}{2 \text{Dimensions}[\text{dataset}][[1] - 1)} \text{Sum}[(\text{dataset}[i, 2] + \text{dataset}[i + 1, 2])(\text{If the interval is constant}), (i, \text{initialpoint}, \text{finalpoint})],\right.$$



$$\text{If}[\text{Length}[\text{Dimensions}[\text{dataset}]] = 2 \ \&\& \ \text{interval} = \text{"VI"}, (1/2) \text{Sum}[(\text{dataset}[i, 2] + \text{dataset}[i + 1, 2])(\text{dataset}[i + 1, 1] - \text{dataset}[i, 1])(\text{If the interval is variable}), (i, \text{initialpoint}, \text{finalpoint})]]](\text{"Trapezoidal Integration Formula. *})$$


```

iii - Integrate Function - Trapezoidal Method

```

In[*]:= (*integrateFunctionTrap[dataset_,interval_(*"C"onstant "I"ntervals or "V"ariables "I"ntervals*)] :=
Table[TrapIntV11[dataset[[1;;i]],interval],{i,Dimensions[dataset][[1]]}*)

In[*]:= (*integrateFunctionTrap2[dataset_, interval_(*"C"onstant "I"ntervals or "V"ariables "I"ntervals*)] :=
Table[Piecewise[{{TrapIntV12[dataset,interval,1,2],i==Length[dataset]},
{TrapIntV11[dataset[[1;;i]],interval],2<=i<=Length[dataset]-1}},{i,Length[dataset]}]*)

```



```

In[109]:=
integrateFunctionTrap[dataset_, initialValues_, intervalType_] :=
Block[{t0 = If[Length[Dimensions[dataset]] == 2, initialValues[[1]], f0 = If[Length[Dimensions[dataset]] == 2, initialValues[[2]],
  (*bloqueia*) (*comprimento*) (*dimensões*) (*comprimento*) (*dimensões*)
  sollist = If[Length[Dimensions[dataset]] == 2, {{initialValues[[1]], initialValues[[2]]}, {initialValues[[1]]},
  (*comprimento*) (*dimensões*)
  tf = If[Length[Dimensions[dataset]] == 2, dataset[[All, 1]][[-1]],
  (*comprimento*) (*dimensões*) (*tudo*)
  ff = If[Length[Dimensions[dataset]] == 2, dataset[[All, 2]][[-1]], dataset[[-1]],
  (*comprimento*) (*dimensões*) (*tudo*)
  dts = dataset, n = 2, tn, fn, told, fold, steps, h, tnew, fnew},
  steps = Length[dts[[All, 1]]] - 1;
  (*comprimento*) (*tudo*)
  (*told=t0;
  fold=f0;*)
  Do[
  (*repete*)
    tnew = dts[[All, 1]][[n]];
    (*tudo*)
    fnew = TrapIntV11[dts[[1 ;; n]], intervalType];
    sollist = Append[sollist, {tnew, fnew}]; (*Mounting the result vector*)
    (*adiciona*)
    n = n + 1, {steps}];
  Return[sollist] (*to see all grid points*)
  (*retorna*)

```

1.2.3.d - The Euler Integration Method

i - Euler by Finite Differences

```

In[96]:= IntEulerV13[dataset_, initialValues_] :=
Block[{t0 = initialValues[[1]], f0 = initialValues[[2]], sollist = {{initialValues[[1]], initialValues[[2]]},
  (*bloqueia*)
  tf = dataset[[All, 1]][[-1]], dts = dataset, n = 2, ff = dataset[[All, 2]][[-1]], tn, fn, told, fold, steps, h, tnew, fnew},
  (*tudo*) (*tudo*)
  (*h=N[(tn-t0)/steps];*)
  (*valor numérico*)
  h = N[(tf - t0)/Length[dts[[All, 1]]];
  (*valor numérico*) (*comprimento*) (*tudo*)
  steps = Length[dts[[All, 1]]] - 1;
  (*comprimento*) (*tudo*)
  told = t0;
  fold = f0;
  (*tn;
  fn;
  *)Do[
  (*repete*)
    tnew = dts[[n - 1, 1]];
    fnew = sollist[[n - 1, 2]] + dts[[n - 1, 2]] (tnew - told);
    sollist = Append[sollist, {tnew, fnew}]; (*Mounting the result vector*)
    (*adiciona*)
    told = tnew;
    n = n + 1
    (*tnew=told+h;(*"FOR" to Wolfram Language*)
    (*idioma*)
    fnew=fold+h*dataset[[n,2]];
    *)(*tnew=tn;(*"FOR" to Wolfram Language*)
    (*idioma*)
    fnew=fn+h*dataset[[n-1,2]];*)
    (*told=tnew;
    fold=fnew*)
    (*tn=tnew;
    fn=fnew*), {steps}];
  Return[sollist] (*to see all grid points*)
  (*retorna*)

```

1.2.3.e - Derivative by Interpolation Polynomials

i - Derivative - Forward - "i"-Derivative

```
In[115]:=
derFor[dataset_, i_] := Block[{sollist, derivative, steps = 1},
  (*If the interval is unitary or not specified*)
  If[Length[Dimensions[dataset]] == 1, steps = Length[dataset],
    (*If the interval is unitary or not specified*)
    If[Length[Dimensions[dataset]] == 2, steps = Length[dataset[[All, 1]]];]
  ];
  Do[
    If[Length[Dimensions[dataset]] == 1, derivative = (dataset[i + 1] - dataset[i]);
      (*If the interval is unitary or not specified*)
    ];
    If[Length[Dimensions[dataset]] == 2,
      derivative = ((dataset[i + 1, 2] - dataset[i, 2]) / (dataset[i + 1, 1] - dataset[i, 1]));
      (*If the interval Xi+1 - Xi is constant*)
    ];
    {steps}];
  Return[derivative] (*to see all grid points*)
]
```

ii - Derivative - Forward - Starting in the first element

```
In[116]:=
derForv2[dataset_] := Block[{sollist = {0}, i = 1, derivative, steps},
  If[Length[Dimensions[dataset]] == 1, steps = Length[dataset],
    If[Length[Dimensions[dataset]] == 2, steps = Length[dataset[[All, 1]]];]
  ];
  Do[
    If[Length[Dimensions[dataset]] == 1, derivative = (dataset[i + 1] - dataset[i]);
      (*If the interval is unitary or not specified*)
    ];
    sollist = Append[sollist, derivative];
    i = i + 1 (*If the interval is unitary or not specified*);

    If[Length[Dimensions[dataset]] == 2, derivative = ((dataset[i + 1, 2] - dataset[i, 2]) / (dataset[i + 1, 1] - dataset[i, 1]));
      (*If the interval Xi+1 - Xi is constant*)
    ];
    sollist = Append[sollist, derivative];
    i = i + 1 (*If the interval Xi+1 - Xi is constant*);

    , {steps - 1}];
  Return[sollist] (*to see all grid points*)
]
```

iii - Derivative - Forward - Starting in the desired element

```

In[117]:=
derForv3[dataset_, InitialElement_, FinalElement_] :=
  Block[
    {sollist = {}, i = 1, IE = InitialElement, FE = FinalElement, derivative, steps},
    If[Length[Dimensions[dataset]] == 1, steps = Length[dataset[[IE ;; FE]]],
      (* compr... dimensões comprimento *)
      If[Length[Dimensions[dataset]] == 2, steps = Length[dataset[[IE ;; FE]][[All, 1]]],
        (* compr... dimensões comprimento tudo *)
        Do[
          (* repete *)
          If[Length[Dimensions[dataset[[IE ;; FE]]]] == 1, derivative = (dataset[[IE ;; FE]][[i + 1]] - dataset[[IE ;; FE]][[i]]);
          (* compr... dimensões *)
          sollist = Append[sollist, derivative];
          (* adiciona *)
          i = i + 1 (*If the interval is unitary or not specified*);
          (* se *)
          If[Length[Dimensions[dataset[[IE ;; FE]]]] == 2, derivative = 
$$\left( \frac{\text{dataset}[[\text{IE} ;; \text{FE}][[i + 1, 2]] - \text{dataset}[[\text{IE} ;; \text{FE}][[i, 2]]]}{\text{dataset}[[\text{IE} ;; \text{FE}][[i + 1, 1]] - \text{dataset}[[\text{IE} ;; \text{FE}][[i, 1]]]} \right);$$

          (* compr... dimensões *)
          sollist = Append[sollist, derivative];
          (* adiciona *)
          i = i + 1 (*If the interval Xi+1 - Xi is constant*);
          (* se *)
          , {steps - 1}];
        Return[sollist] (*to see all grid points*)
      ]
    ]
  ]
  (* retorna *)

```

iv - Derivative Function - Forward

```

In[118]:=
derivativeFunctionv1[dataset_, IE_, FE_] :=
  Table[
    Piecewise[
      {
        {derForv3[dataset, IE, i_Integer], i <= Length[dataset] - 1},
        (* tabela função por partes número inteiro comprimento *)
        {derBackv6[dataset, i_Integer, FE], i == Length[dataset]}],
      {i, Length[dataset]}],
    (* número inteiro comprimento comprimento *)

```

v - Derivative - Backward - “i”-Derivative

```

In[119]:=
derBack[dataset_, i_] := Block[
  {sollist = {0}, derivative, steps = 1},
  (* bloqueia *)
  (*If[Length[Dimensions[dataset]]==1,steps=Length[dataset],
    (* compr... dimensões comprimento *)
    If[Length[Dimensions[dataset]]==2,steps=Length[dataset[[All,1]]];*)
    (* compr... dimensões comprimento tudo *)
  Do[
    (* repete *)
    If[Length[Dimensions[dataset]] == 1, derivative = (dataset[[i]] - dataset[[i - 1]]);(*If the interval is unitary or not specified*);
    (* compr... dimensões *)
    If[Length[Dimensions[dataset]] == 2,
      (* compr... dimensões *)
      derivative = 
$$\left( \frac{\text{dataset}[[i, 2]] - \text{dataset}[[i - 1, 2]]}{\text{dataset}[[i, 1]] - \text{dataset}[[i - 1, 1]]} \right);$$

      (*If the interval Xi+1 - Xi is constant*);
      (* se *)
      , {steps}];
    Return[derivative] (*to see all grid points*)
  ]
  (* retorna *)

```

vi - Derivative - Backward - Starting in the second element

```

In[120]:=
derForv5[dataset_] := Block[{sollist = {}, i = 2, derivative, steps},
    (*bloqueia*)

    If[Length[Dimensions[dataset]] == 1, steps = Length[dataset],
    (*comprimento*)
    (*dimensões*)

    If[Length[Dimensions[dataset]] == 2, steps = Length[dataset][All, 1]]];
    (*tudo*)
    Do[
    (*repete*)

    If[Length[Dimensions[dataset]] == 1, derivative = (dataset[[i]] - dataset[[i - 1]]);
    (*dimensões*)
    (*comprimento*)
    sollist = Append[sollist, derivative];
    (*adiciona*)

    i = i + 1 (*If the interval is unitary or not specified*);
    (*se*)

    If[Length[Dimensions[dataset]] == 2, derivative =  $\left( \frac{\text{dataset}[[i, 2]] - \text{dataset}[[i - 1, 2]]}{\text{dataset}[[i, 1]] - \text{dataset}[[i - 1, 1]]} \right)$ ;
    (*dimensões*)
    (*comprimento*)

    sollist = Append[sollist, derivative];
    (*adiciona*)

    i = i + 1 (*If the interval Xi+1 - Xi is constant*)
    (*se*)

    , {steps}];

    Return[sollist] (*to see all grid points*)
    (*retorna*)

```

vii - Derivative - Backward - Starting in the desired element

```

In[121]:=
derBackv6[dataset_, InitialElement_, FinalElement_] :=
Block[{sollist = {}, i = 2, IE = InitialElement, FE = FinalElement, derivative, steps},
    (*bloqueia*)

    If[Length[Dimensions[dataset]] == 1, steps = Length[dataset][IE ;; FE]],
    (*comprimento*)
    (*dimensões*)

    If[Length[Dimensions[dataset]] == 2, steps = Length[dataset][IE ;; FE][All, 1]]];
    (*tudo*)
    Do[
    (*repete*)

    If[Length[Dimensions[dataset][IE ;; FE]] == 1, derivative = (dataset[[IE ;; FE]][i] - dataset[[IE ;; FE]][i - 1]);
    (*dimensões*)
    (*comprimento*)
    sollist = Append[sollist, derivative];
    (*adiciona*)

    i = i + 1 (*If the interval is unitary or not specified*);
    (*se*)

    If[Length[Dimensions[dataset][IE ;; FE]] == 2, derivative =  $\left( \frac{\text{dataset}[[IE ;; FE]][i, 2] - \text{dataset}[[IE ;; FE]][i - 1, 2]]{\text{dataset}[[IE ;; FE]][i, 1] - \text{dataset}[[IE ;; FE]][i - 1, 1]]} \right)$ ;
    (*dimensões*)
    (*comprimento*)

    sollist = Append[sollist, derivative];
    (*adiciona*)

    i = i + 1 (*If the interval Xi+1 - Xi is constant*)
    (*se*)

    , {steps}];

    Return[sollist] (*to see all grid points*)
    (*retorna*)

```

viii - Derivative Function - Backward

```

In[122]:=
derivativeFunctionv2[dataset_, IE_, FE_] := Table[Piecewise[{{derForv3[dataset, IE, i_Integer], i < 2},
    (*tabela*) (*função por partes*) (*número inteiro*)

    {derBackv6[dataset, i_Integer, FE], 2 ≤ i ≤ Length[dataset]}], {i, Length[dataset]}]
    (*número inteiro*) (*comprimento*) (*comprimento*)

```

1.2.3.f - Time's Formula

```
In[123]:=
(*Δt[n_]:=time[[n+1]]-time[[n]]*)

In[124]:=
Δt[n_]:=time[[n]]-time[[n-1]]
```

1.2.3.g - “u” and “v” Transformation Formulas

```
uvelocity[n_, dataset_, ratio_] := ratio ((dataset[[1, n]] + dataset[[2, n]])/2)
ωvelocity[n_, dataset_, ratio_, distance_] := ratio ((dataset[[1, n]] - dataset[[2, n]])/distance)

In[125]:=
uvelocity[n_, dataset_, ratio_] := ratio ((Evaluate[dataset[[1, n]] + dataset[[2, n]]])/2)
|calcula

In[126]:=
ωvelocity[n_, dataset_, ratio_, distance_] := ratio ((Evaluate[dataset[[1, n]] - dataset[[2, n]]])/distance)
|calcula
```

1.2.3.h - Kinematic Modeling

i. “Psi” Modeling

```
In[* ]:=
ψ[n_] := psi[[n - 1]] + ω[[n]] Δt[n]

In[127]:=
(*ψv1[n_Integer,psiinicial_]:=If[n-1>0,psi[[n-1]]+ω[[n]] Δt[n],psiinicial]*)
|número inteiro |se

In[128]:=
ψv2[time_, psiinicial_] :=
Block[{t = time, psi0 = psiinicial, sollist = {{0, psiinicial}}, omega = ωsignal[All, 2], psitemp, nold = 1, nnew, steps},
|bloqueia |judo
steps = Length[t] - 1;
|comprimento
Do[
|repete
nnew = nold + 1;
psitemp = sollist[[nnew - 1, 2]] + omega[[nnew]] × Δt[nnew];
sollist = Append[sollist, {t[[nold]], psitemp}];
|adiciona
nold = nnew, {nnew = steps}
];
Return[sollist]
|retorna
```

ii. Transformation Matrix

```
In[129]:=
transformationMatrix[psi_, parameterA_] := {{Cos[psi], -parameterA Sin[psi]}, {Sin[psi], parameterA Cos[psi]}, {0, 1}}
|cosseno |seno |seno |cosseno

In[130]:=
(*globalvelocityv1[translationalvelocitysignal_, rotationalvelocitysignal_, psi_, parameterA_] :=
transformationMatrix[psi, parameterA].{translationalvelocitysignal, rotationalvelocitysignal}*)
```

iii. Full Kinematic Modeling

```
In[131]:=
globalvelocityv2[translationalvelocitysignal_, rotationalvelocitysignal_, psi_, parameterA_] :=
Table[transformationMatrix[psi[[n]], parameterA].{translationalvelocitysignal[[n]], rotationalvelocitysignal[[n]]}, {n, Length[psi]}]
|tabela |comprimento

In[132]:=
(*globalvelocityv3[translationalvelocitysignal_, rotationalvelocitysignal_, psi_, parameterA_, n_(*Desired element*)] :=
transformationMatrix[psi[[n]], parameterA].{translationalvelocitysignal[[n]], rotationalvelocitysignal[[n]]}*)
```

iii. Others Modeling

```
In[* ]:=
ψ[n_] := psi[[n - 1]] + ω[[n]] Δt[n]

In[133]:=
(*ψv1[n_Integer,psiinicial_]:=If[n-1>0,psi[[n-1]]+ω[[n]] Δt[n],psiinicial]*)
|número inteiro |se
```

```

In[134]:=
(*ψv2[time_,psiinicial_]:=Block[
    |bloqueia
    {t=time,psi0=psiinicial, sollist={{0,psi0}},omega=ωsignal[All,2],psitemp,nold=1,nnew,psiFin,steps},
    |tudo

    steps = Length[t]-1;
    |comprimento

    Do[nnew=nold+1;
    |repete
        psitemp = sollist[[nnew-1,2]]+omega[[nnew]] Δt[nnew];
        sollist = Append[sollist, {t[[nold]],psitemp}];
        |adiciona
        nold=nnew,{nnew=steps}];
    Return[sollist]]*)
    |retorna

```

1.2.3.i - Error Modeling

“xi” will be used like “Etai” in all “OhErrorType” Functions.

i - Error to Rectangle and Euler Methods

```

In[42]:= OhErrorType1[dataset_, TypeSolution_(*"P"artials or "T"otal*)] := Module[
    |módulo de código
    {sollist = {0}, i = 1, error, steps},

    If[Length[Dimensions[dataset]] == 1, steps = Length[dataset],
    |comprimento |dimensões |comprimento
        If[Length[Dimensions[dataset]] == 2, steps = Length[dataset[All, 1]]];
        |comprimento |dimensões |comprimento |tudo

    Do[
    |repete
        If[Length[Dimensions[dataset]] == 1, If[TypeSolution == "P", error = (dataset[i + 1] - dataset[i])/2,
        |comprimento |dimensões |se
            If[TypeSolution == "T", error = Sum[(dataset[i + 1] - dataset[i])/2, {n, i}]]];
            |se |soma

        (*sollist=Append[sollist,error];
        |adiciona

        i=i+1 (*If the interval is unitary or not specified*)*)
        |se If[Length[Dimensions[dataset]] == 2, If[TypeSolution == "P",
        |comprimento |dimensões |se
            error = (1/2)((dataset[i + 1, 1] - dataset[i, 1])^2)
            |se
                ((dataset[i + 1, 2] - dataset[i, 2]) / (dataset[i + 1, 1] - dataset[i, 1])),
                If[TypeSolution == "T",
                |se
                    error = Sum[
                    |soma
                        (1/2)((dataset[i + 1, 1] - dataset[i, 1])^2)
                        |se
                            ((dataset[i + 1, 2] - dataset[i, 2]) / (dataset[i + 1, 1] - dataset[i, 1])),
                            {n, i}]]];

        sollist = Append[sollist, error];
        |adiciona

        i = i + 1(*If the interval Xi+1 - Xi is constant*)
        |se

        , {steps - 1}];

    Return[sollist](*to see all grid points*)
    |retorna

```

ii - Error to Center Point Method

```

In[43]:= OhErrorType2[dataset_, TypeSolution_(*"P"artials or "T"otal*)] := Module[{sollist = {0}, i = 1, error, steps},
↳ compr... dimensões comprimento
  If[Length[Dimensions[dataset]] = 1, steps = Length[dataset],
↳ compr... dimensões comprimento tudo
    If[Length[Dimensions[dataset]] = 2, steps = Length[dataset][All, 1]]];
Do
repete
  If[Length[Dimensions[dataset]] = 1, If[TypeSolution == "P", error = (dataset[[i] - 2 dataset[[i + 1]] + dataset[[i + 2]]),
↳ compr... dimensões se
    If[TypeSolution == "T", error = Sum[(dataset[[i] - 2 dataset[[i + 1]] + dataset[[i + 2]]), {n, i}]]];
se soma
    (*sollist=Append[sollist,error];
adiciona
    i=i+1 (*If the interval is unitary or not specified*);*)
se
  If[Length[Dimensions[dataset]] = 2, If[TypeSolution == "P", error = (3/4)((dataset[[i + 1, 1]] - dataset[[i, 1]])^3)
↳ compr... dimensões se
    
$$\left( \frac{(\text{dataset}[[i, 2]] - 2 \text{dataset}[[i + 1, 2]] + \text{dataset}[[i + 2, 2]])}{(\text{dataset}[[i + 1, 1]] - \text{dataset}[[i, 1]])^2} \right), \text{If}[\text{TypeSolution} == "T", \text{error} =$$


$$\text{Sum}\left[\left(\frac{3}{4}\right)((\text{dataset}[[i + 1, 1]] - \text{dataset}[[i, 1]])^3) \left( \frac{(\text{dataset}[[i, 2]] - 2 \text{dataset}[[i + 1, 2]] + \text{dataset}[[i + 2, 2]])}{(\text{dataset}[[i + 1, 1]] - \text{dataset}[[i, 1]])^2} \right), \{n, i\}]]];$$

soma
    sollist = Append[sollist, error];
adiciona
    i = i + 1 (*If the interval Xi+1 - Xi is constant*)
se
    , {steps - 2}];
  Return[sollist] (*to see all grid points*)
retorna

```

iii - Error to Trapezoid Method

```

In[44]:= OhErrorType3[dataset_, TypeSolution_(*"P"artials or "T"otal*)] := Module[(*módulo de código*) {sollist = {0}, i = 1, error, steps},

  If[Length[Dimensions[dataset]] == 1, steps = Length[dataset],
(*Comprimento*)
(*dimensões*)
(*comprimento*)
(*tudo*)
    If[Length[Dimensions[dataset]] == 2, steps = Length[dataset][[All, 1]]];

  Do[
(*repete*)
    If[Length[Dimensions[dataset]] == 1,
(*Comprimento*)
(*dimensões*)
      If[TypeSolution == "P", error = -(1/12)(dataset[[i]] + 6 dataset[[i + 2]] - 4 dataset[[i + 3]] + dataset[[i + 4]]),
(*se*)
      If[TypeSolution == "T", error = Sum[(*soma*) -(1/12)(dataset[[i]] + 6 dataset[[i + 2]] - 4 dataset[[i + 3]] + dataset[[i + 4]]), {n, i}]]];
(*adiciona*)
    sollist = Append[sollist, error];

    i = i + 1; (*se*) (*If the interval is unitary or not specified*);

    If[Length[Dimensions[dataset]] == 2, If[TypeSolution == "P", error = -(1/12)((dataset[[i + 1, 1]] - dataset[[i, 1]])^2)
(*Comprimento*)
(*dimensões*)
(*se*)
      (

$$\frac{(\text{dataset}[[i, 2]] + 6 \text{dataset}[[i + 2, 2]] - 4 \text{dataset}[[i + 3, 2]] + \text{dataset}[[i + 4, 2]])}{(\text{dataset}[[i + 1, 1]] - \text{dataset}[[i, 1]])^4}$$

),
      If[TypeSolution == "T", error = Sum[(*soma*) -(1/12)((dataset[[i + 1, 1]] - dataset[[i, 1]])^2)
(*se*)
        (

$$\frac{(\text{dataset}[[i, 2]] + 6 \text{dataset}[[i + 2, 2]] - 4 \text{dataset}[[i + 3, 2]] + \text{dataset}[[i + 4, 2]])}{(\text{dataset}[[i + 1, 1]] - \text{dataset}[[i, 1]])^4}$$

), {n, i}]]];
(*adiciona*)
    sollist = Append[sollist, error];

    i = i + 1; (*se*) (*If the interval Xi+1 - Xi is constant*)

    , {steps - 4}];

  Return[sollist] (*retorna*) (*to see all grid points*)]

```

XX - Tests

```

In[45]:= teste = {{0, 2}, {3, 5}, {4, 7}, {8, 11}}

Out[45]=
{{0, 2}, {3, 5}, {4, 7}, {8, 11}}

In[69]:= teste2 = {{3, 3}, {5, 6}, {7, 4}, {9, 8}, {11, 12}}

Out[69]=
{{3, 3}, {5, 6}, {7, 4}, {9, 8}, {11, 12}}

In[110]:= teste3 = {{0, 1}, {0.25, 1.064494}, {0.5, 1.284025}, {0.75, 1.755055}, {1, 2.718281},
  {1.25, 4.770733}, {1.5, 9.487736}, {1.75, 21.380943}, {2, 54.598150}};

In[49]:= Total[(*total*) integrateFunctionRightRect[teste, "VI"] [[2 ;; 4]] // N (*valor numérico*)

In[107]:= integrateFunctionTrap[teste2, {3, 0}, "CI"]

Out[107]=
{{3, 0}, {5, 9}, {7, 19}, {9, 31}, {11, 51}}

In[68]:= integrateFunctionRightRect[teste, "VI"] // N (*valor numérico*)

Out[68]=
{0., 15., 22., 66.}

```



```

In[111]:=
  integrateFunctionTrap[teste3, {0, 0}, "CI"]

Out[111]=
  {{0, 0}, {0.25, 0.258062}, {0.5, 0.551627}, {0.75, 0.931512},
   {1, 1.49068}, {1.25, 2.42681}, {1.5, 4.20911}, {1.75, 8.0677}, {2, 17.5651}}

In[51]:= RightRectIntV3[teste, "VI"]

Out[51]=
  66

In[84]:= (teste2[[-1, 1]] - teste2[[1, 1]]) / 4 // N
                                         |valor numérico

Out[84]=
  2.

In[83]:= Dimensions[teste2][[1]]
         |dimensões

Out[83]=
  5

integrateFunctionRightRect
integrateFunctionLeftRect
integrateFunctionTrap
IntEulerV13

```

2 - Assignments

Preview...

So, realize the follow steps:

step 1 - Realize the charge to dataset...

1. Realize a leitura do arquivo de dados “*wRwL.txt*”, o qual possui 400 linhas e 3 colunas. Em outras palavras, o arquivo contém 400 medidas de tempo, velocidade da roda direita “*wR*” e velocidade da roda esquerda “*wL*”.
2. Para demonstrar a evolução temporal das velocidades das rodas, plote os gráficos que relaciona o “*tempo x wR*” e o “*tempo x wL*”.
3. Transforme as velocidades “*wR*” e “*wL*” em sinais de controle “*u*” e “*ω*”.

- Velocidade linear: $u = r \frac{\omega_R + \omega_L}{2}$
- Velocidade angular: $\omega = r \frac{\omega_R - \omega_L}{d}$

4. Para demonstrar a evolução temporal dos sinais de controle do robô, plote os gráficos que relacionam o “*tempo x u*” e o “*tempo x ω*”.
5. Use o modelo cinemático, para determinar a posição e orientação do robô ao longo do tempo. A título de exemplo, para determinar a orientação do robô, faça:

$$\dot{\psi}(k) = [\psi(k) - \psi(k-1)] / \Delta t = \omega,$$

ou ainda:

$$\psi(k) = \psi(k-1) + \omega \Delta t.$$

6. Para demonstrar a evolução temporal da posição e orientação do robô, plote os gráficos que relaciona o “*tempo x x*” e o “*tempo x y*”, o “*tempo x ψ*”.
7. Para demonstrar a navegação do robô no plano cartesiano, plote o gráfico que relaciona *x* e *y*.

8. Informe numericamente a posição final do robô ao final do experimento.

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\psi} \end{bmatrix} = \begin{bmatrix} \cos \psi & -a \sin \psi \\ \sin \psi & a \cos \psi \\ 0 & 1 \end{bmatrix} \begin{bmatrix} u \\ \omega \end{bmatrix}$$

sabendo que " a " = 0,10 m.

2.1 - Data Import, Channels distribution and Plot signals:

2.1.1 - Data import:

1. Realize a leitura do arquivo de dados " $wRwL.txt$ ", o qual possui 400 linhas e 3 colunas. Em outras palavras, o arquivo contém 400 medidas de tempo, velocidade da roda direita " ωR " e velocidade da roda esquerda " ωL ".

```
In[135]:=
DataSample =
  (*Import["F:\trab_30MAR11\dados_Fernando\12032011_125559\Voltage.txt", "Table"];*)
  Import[
    ImportingFile["W", "Table", "Projeto_Final", "01_Arqs_de_base", "wRwL", txt];
    (*amostraA=Drop[amostra,7];*)
    SizeSample = Dimensions[DataSample]
    Out[136]=
    {400, 3}
```

2.1.2 - Organizing the channels and samples:

```
In[137]:=
Sample1 = DataSample[All, 1];(* Incremental time*)
time = DataSample[All, 1];(* Incremental time*)

In[139]:=
(*time=Sample1;*)

In[140]:=
(*Sample2=DataSample[All,2];(* ωR *)*)

In[141]:=
(*ωRigth=Sample2;(* ωR *)*)
ωRigth = DataSample[All, 2];(* ωR *)

In[142]:=
(*Sample3=DataSample[All,3];(* ωL *)*)

In[143]:=
(*ωLeft=Sample3;*)
ωLeft = DataSample[All, 3];(* ωL *)

In[144]:=
SampleVerify1 = Transpose[{time, ωRigth];(* Δt vs ωR *)

In[145]:=
(*SampleVerify1= Transpose[{Sample1,Sample2}];(* Δt vs ωR *)*)

In[146]:=
SampleVerify2 = Transpose[{time, ωLeft];(* Δt vs ωL *)
```

```

In[147]:=
(*SampleVerify2= Transpose[{Sample1,Sample3}];(* Δt vs ωL *)*)
      ↳transposição

In[148]:=
name1 = "ωR";(* Δt vs ωR *)

In[149]:=
name2 = "ωL";(* Δt vs ωL *)

In[150]:=
names3 = {"ωR", "ωL"};(* ωR and ωL *)

In[151]:=
(*names1= {"time", "ωR"};(* Δt vs ωR *)*)

In[152]:=
(*names2= {"time", "ωL"};(* Δt vs ωL *)*)

```

2.1.3 - Plot signals:

By calculating a derivative in the time signal, it is possible verify that the time signal constitutes a linear data set.

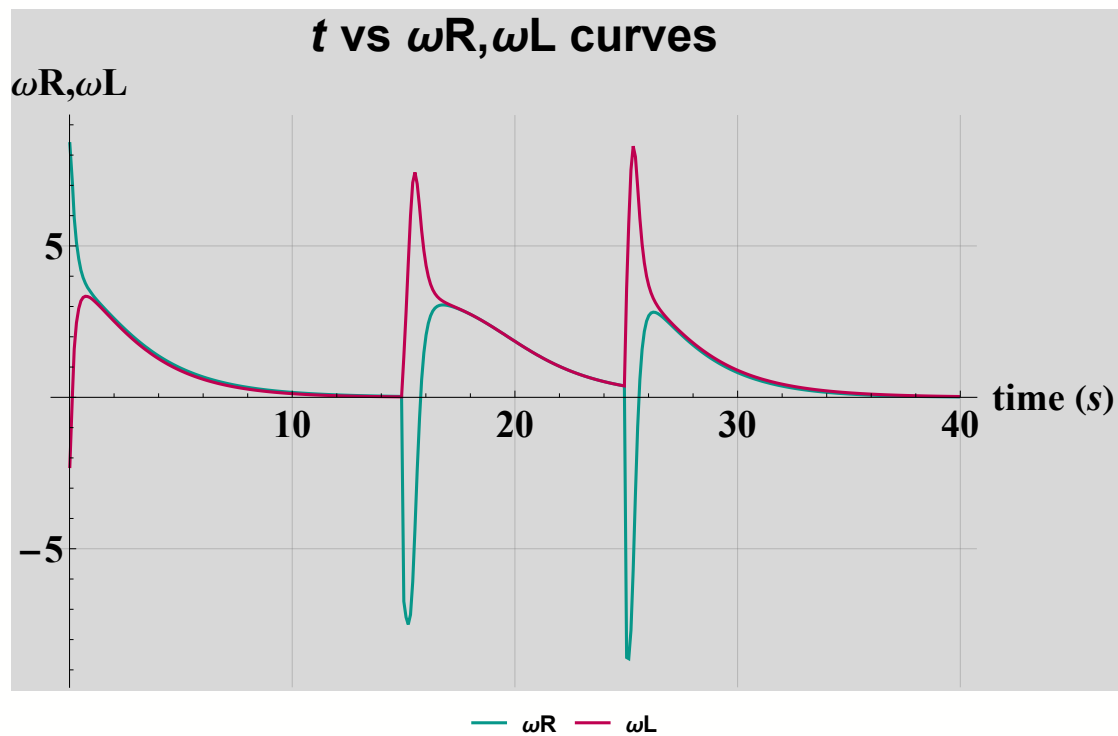
2. Para demonstrar a evolução temporal das velocidades das rodas, plote os gráficos que relaciona o “*tempo x ωR*” e o “*tempo x ωL*”.

```

In[153]:=
graf1 = graphs1[{SampleVerify1, SampleVerify2}, names3]

```

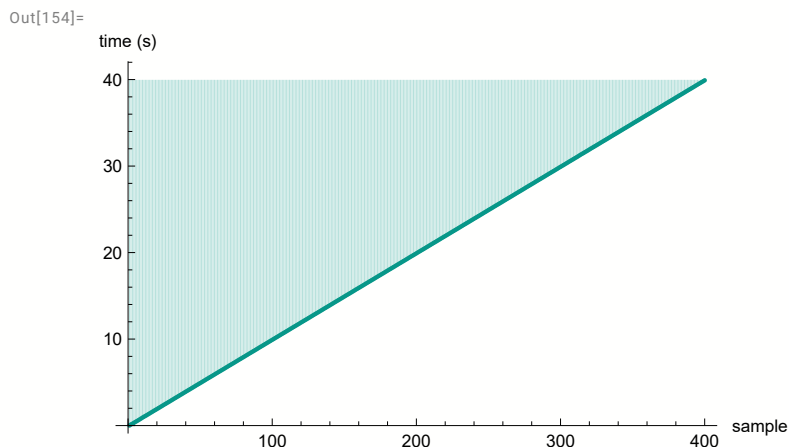
Out[153]=



2.1.4 - Verifying the time channel:

By calculating a derivative in the time signal, it is possible verify that the time signal constitutes a linear data set.

```
In[154]:= ListPlot[time, AxesLabel → {"sample", "time (s)"}, Filling → Top]
          gráfico de uma ... | legenda dos eixos | coloração | topo
```



Furthermore, by performing a more detailed analysis of time signal, it is possible to concluded that of the time signal evolves linearly due to the sampling frequency, possibly.

To calculate of the time interval Δt , it is calculated by subtracting of $t(n+1)$ and $t(n)$.

So,

$$\Delta t = t[[n+1]] - t[[n]]$$

The mean time interval Δt_{mean} is defined by the calculate of the mean time interval. So:

```
In[155]:= tMeanDataSet = Table[Δt[n], {n, 2, Length[time]};
          tabela | comprimento
```

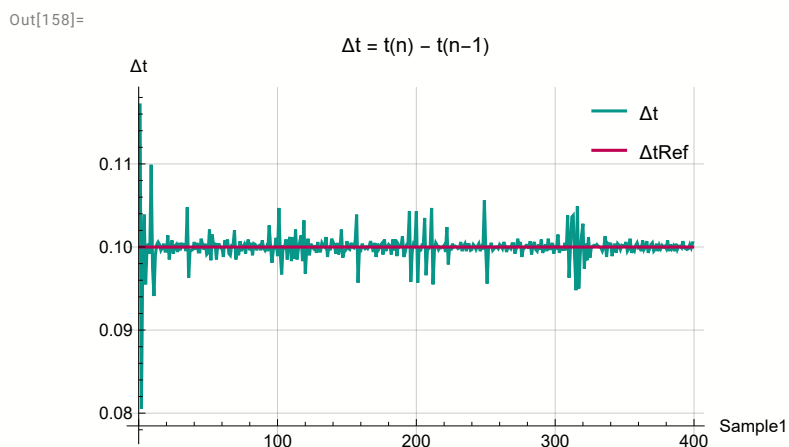
```
In[156]:= ΔtMean = Mean[tMeanDataSet];
          média
```

where the “AverRefTimeInterval” is the average reference time interval.

```
In[157]:= AverRefTimeInterval = Table[ΔtMean, {Length[tMeanDataSet]};
          tabela | comprimento
```

Plotting this results in the “GrafTimeVariation” graphics, occurs:

```
In[158]:= GrafTimeVariation = ListPlot[{tMeanDataSet, AverRefTimeInterval},
          gráfico de uma lista de valores
          Joined → True, PlotRange → All, GridLines → Automatic, AxesLabel → {"Sample1", "Δt"},
          |unido | ver... | intervalo do g... | t... | grade de lin... | automático | legenda dos eixos
          PlotLabel → "Δt = t(n) - t(n-1)", PlotLegends → Placed[{"Δt", "ΔtRef"}, {Right, Top}]
          |etiqueta de gráfico | legenda do gráf... | situado | direita | topo
```



2.2 - Calculating and verifying the “ u ” and “ v ” control signals:

Prologue

Considering two wheels with $r = rR = rL = 0,1$ meters and the distance “ d ” between the wheels being $0,4$ meters, and transforming the velocity signals “ ωR ” and “ ωL ” into control signals “ u ” and “ ω ”, it follows that:

$$uvelocity[n_ , dataset_ , ratio_] := ratio ((dataset[[1, n]] + dataset[[2, n]])/2)$$

$$\omega velocity[n_ , dataset_ , ratio_ , distance_] := ratio ((dataset[[1, n]] - dataset[[2, n]])/distance)$$

The “ u ” and “ ω ” signals can be organizing in a “ $u\omega$ signals” matrix. So:

```
In[159]:=
{r, d, a} = {0.1, 0.4, 0.1};

In[160]:=
uωsignals =
  Transpose[Table[{uvelocity[n, {ωRigth, ωLeft}, r], ωvelocity[n, {ωRigth, ωLeft}, r, d]}, {n, Length[time]}]];
  ↳transposição ↳tabela                                  ↳comprimento

In[161]:=
Dimensions[uωsignals]
↳dimensões

Out[161]=
{2, 400}

In[162]:=
usignal = Transpose[{time, uωsignals[[1]]}];
  ↳transposição
usignal // Dimensions
  ↳dimensões
ωsignal = Transpose[{time, uωsignals[[2]]}];
  ↳transposição
ωsignal // Dimensions
  ↳dimensões

Out[163]=
{400, 2}

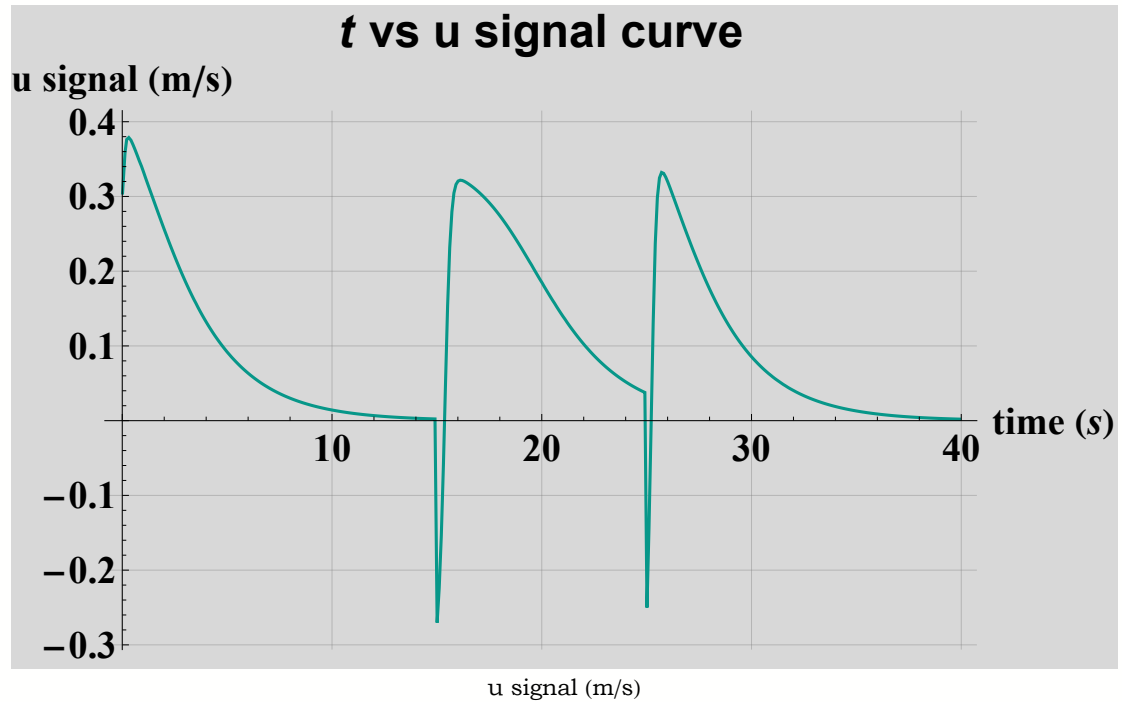
Out[165]=
{400, 2}
```

Plotting the “ $time$ vs u ” and “ $time$ vs ω ” graphics, occurs:

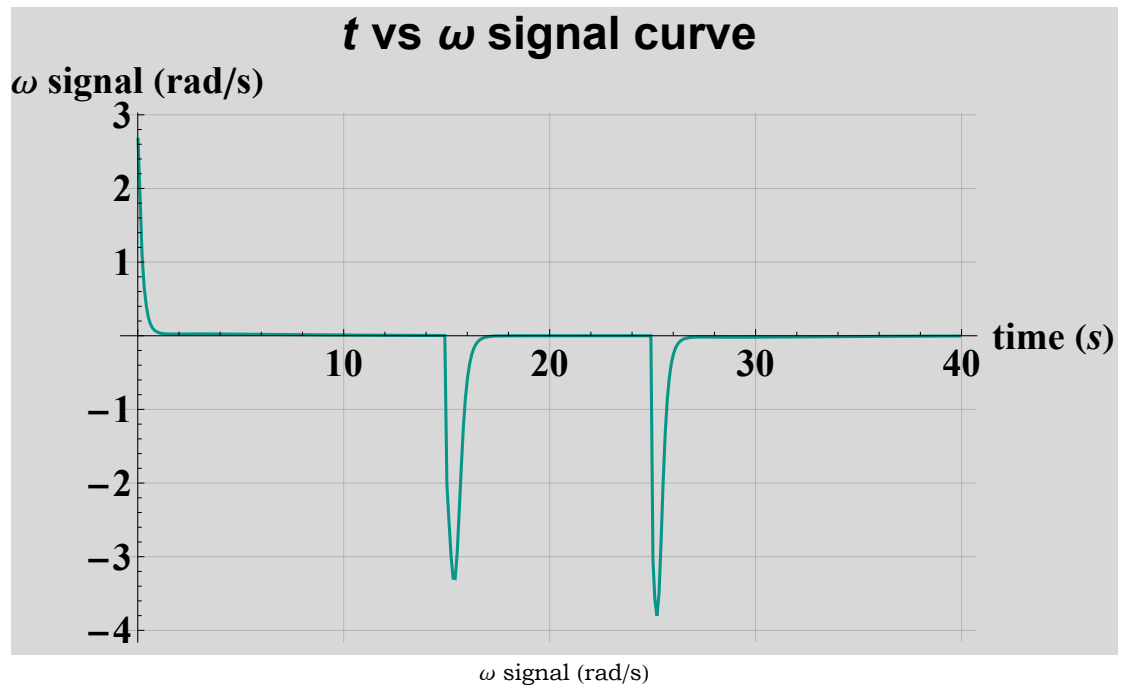
```
graphs1[usignal, "u signal"]
graphs1[ωsignal, "ω signal"]
```

```
In[166]:=
graphs2[usignal, "u signal", "m/s"]
graphs2[ωsignal, "ω signal", "rad/s"]
```

```
Out[166]=
```



```
Out[167]=
```



It is assumed that the robot starts from initial position $(x_0, y_0) = (0, 0)$, with pose $\psi_0 = 0 \text{ rad}$.

2.3 - Orientation and Position:

2.3.1 - Orientation:

5. Use o modelo cinemático, para determinar a posição e orientação do robô ao longo do tempo. A título de exemplo, para determinar a orientação do robô, faça:

$$\dot{\psi}(k) = [\psi(k) - \psi(k-1)]/\Delta t = \omega,$$

ou ainda:

$$\psi(k) = \psi(k-1) + \omega \Delta t.$$

```
In[168]:=
  ω = ωsignal;

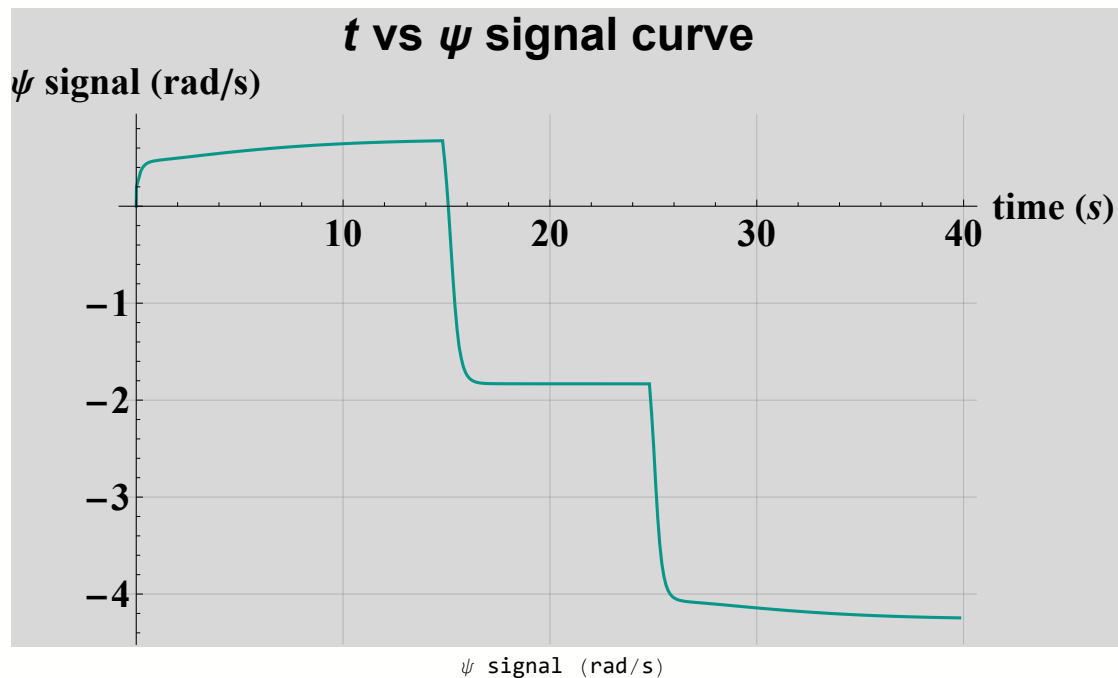
In[169]:=
  ψSamples = ψv2[time, 0][All, 2];

In[170]:=
  ψSamples // Dimensions

Out[170]=
  {400}

In[228]:=
  graphs2[ψv2[time, 0], "ψ signal", "rad/s"]

Out[228]=
```



2.1.4 - Position:

The Dataset for calculations

```
In[*]:= (*globalveloc=globalvelocityv2[usignal[All,2],ωsignal[All,2],ψSamples,a];*)
```

i. The Global Velocities Dataset for calculations

```
In[236]:=
  globalvelocRigthRect = globalvelocityv2[usignal[All, 2], ωsignal[All, 2], integrateFunctionRightRect[ψpto, {0, 0}, "VI"][All, 2], a];

In[237]:=
  globalvelocLeftRect = globalvelocityv2[usignal[All, 2], ωsignal[All, 2], integrateFunctionLeftRect[ψpto, {0, 0}, "VI"][All, 2], a];

In[238]:=
  globalvelocTrap = globalvelocityv2[usignal[All, 2], ωsignal[All, 2], integrateFunctionTrap[ψpto, {0, 0}, "VI"][All, 2], a];

In[239]:=
  globalvelocEuler = globalvelocityv2[usignal[All, 2], ωsignal[All, 2], IntEulerV13[ψpto, {0, 0}][All, 2], a];
```

ii. The Global Velocities Vectors for calculations

```
In[173]:=
xpto = Transpose[{time, globalveloc[All, 1]]};(* In function of "t" *)
      |transposição      |judo      |entrada
ypto = Transpose[{time, globalveloc[All, 2]]};(* In function of "t" *)
      |transposição      |judo      |entrada
ψpto = Transpose[{time, globalveloc[All, 3]]};(* In function of "t" *)
      |transposição      |judo      |entrada

In[248]:=
xptoRRect = Transpose[{time, globalvelocRigthRect[All, 1]]};(* In function of "t" *)
            |transposição      |judo      |entrada
yptoRRect = Transpose[{time, globalvelocRigthRect[All, 2]]};(* In function of "t" *)
            |transposição      |judo      |entrada
ψptoRRect = Transpose[{time, globalvelocRigthRect[All, 3]]};(* In function of "t" *)
            |transposição      |judo      |entrada

In[260]:=
xptoLRect = Transpose[{time, globalvelocLeftRect[All, 1]]};(* In function of "t" *)
            |transposição      |judo      |entrada
yptoLRect = Transpose[{time, globalvelocLeftRect[All, 2]]};(* In function of "t" *)
            |transposição      |judo      |entrada
ψptoLRect = Transpose[{time, globalvelocLeftRect[All, 3]]};(* In function of "t" *)
            |transposição      |judo      |entrada

In[254]:=
xptoTrap = Transpose[{time, globalvelocTrap[All, 1]]};(* In function of "t" *)
            |transposição      |judo      |entrada
yptoTrap = Transpose[{time, globalvelocTrap[All, 2]]};(* In function of "t" *)
            |transposição      |judo      |entrada
ψptoTrap = Transpose[{time, globalvelocTrap[All, 3]]};(* In function of "t" *)
            |transposição      |judo      |entrada

In[257]:=
xptoEu = Transpose[{time, globalvelocEuler[All, 1]]};(* In function of "t" *)
          |transposição      |judo      |entrada
yptoEu = Transpose[{time, globalvelocEuler[All, 2]]};(* In function of "t" *)
          |transposição      |judo      |entrada
ψptoEu = Transpose[{time, globalvelocEuler[All, 3]]};(* In function of "t" *)
          |transposição      |judo      |entrada
```

a - Integrating by Right Rectangle Method

a.1 - Right Rectangle

```
In[263]:=
xglobA1 = integrateFunctionRightRect[xptoRRect, {0, 0}, "VI"];
yglobA2 = integrateFunctionRightRect[yptoRRect, {0, 0}, "VI"];
ψglobA3 = integrateFunctionRightRect[ψptoEu, {0, 0}, "VI"];
solBackRectMethod = {time, xglobA1, yglobA2, ψglobA3};
```

a.2 - Right Rectangle Method Error

a.3 - Graphics

b - Integrating by Center Point Method

b.1 - Center Point Method

b.2 - Center Point Method Error

b.3 - Graphics

c - Integrating by Left Rectangle Method

c.1 - Left Rectangle

```
In[271]:=
xglobC13 = integrateFunctionLeftRect[xptoLRect, {0, 0}, "VI"];
yglobC14 = integrateFunctionLeftRect[yptoLRect, {0, 0}, "VI"];
ψglobC15 = integrateFunctionLeftRect[ψptoLRect, {0, 0}, "VI"];
solRectForwMethod = {time, xglobC13, yglobC14, ψglobC15};
```

c.2 - Left Rectangle Method Error

c.3 - Graphics

d - Integrating by Trapezoidal Method**d.1 - Trapezoidal Method**

```
In[279]:=
xglobD19 = integrateFunctionTrap[xptoTrap, {0, 0}, "VI"];
yglobD20 = integrateFunctionTrap[yptoTrap, {0, 0}, "VI"];
ψglobD21 = integrateFunctionTrap[ψptoTrap, {0, 0}, "VI"];
solBackTrapMethod = {time, xglobD19, yglobD20, ψglobD21};
```

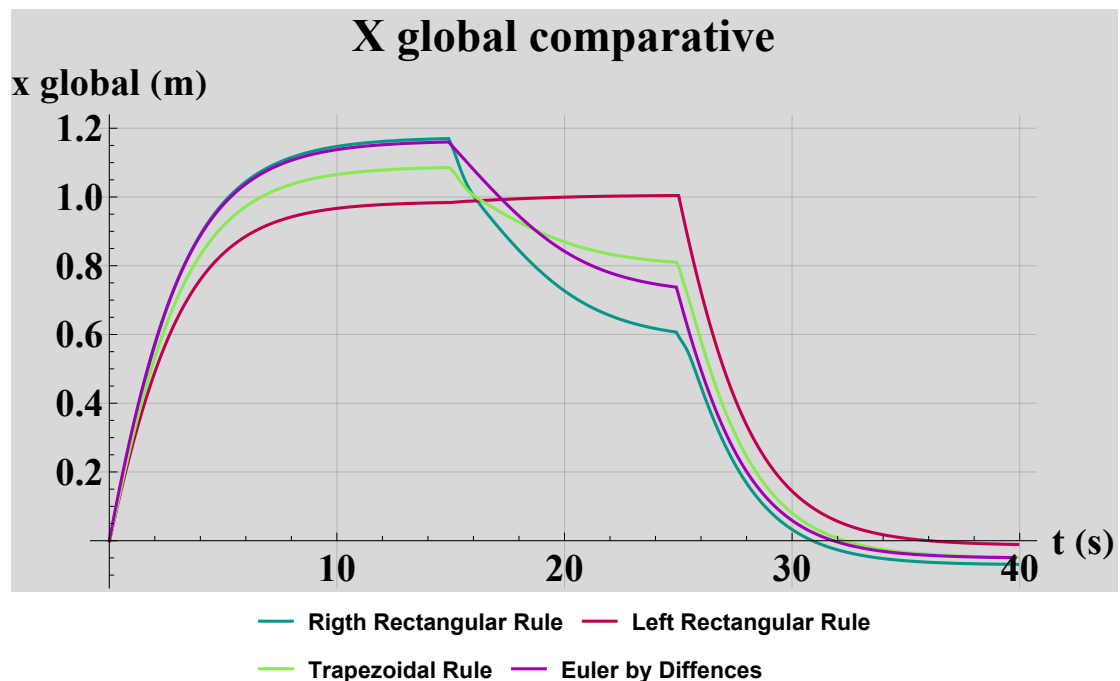
d.2 - Trapezoidal Method Error**d.3 - Graphics****d - Integrating by NONONO Trapezoidal Method****e - Integrating by Euler Method (Problem of Initial Value)****e.1 - Euler Method**

```
In[287]:=
xglobF31 = IntEulerV13[xptoEu, {0, 0}];
yglobF32 = IntEulerV13[yptoEu, {0, 0}];
ψglobF33 = IntEulerV13[ψptoEu, {0, 0}];
solEulerMethod = {time, xglobF31, yglobF32, ψglobF33};
```

e.2 - Euler Method Error**e.3 - Graphics****2.1.4 - Results:**

```
In[320]:=
grafXglobal = graphs5[{xglobA1, xglobC13, xglobD19, xglobF31},
  "X global comparative", {"Rigth Rectangular Rule", "Left Rectangular Rule",
    lregra lresquerda lregra
    "Trapezoidal Rule", "Euler by Diffences"}, {"t", "x global"}, {"s", "m"}]
lregra
```

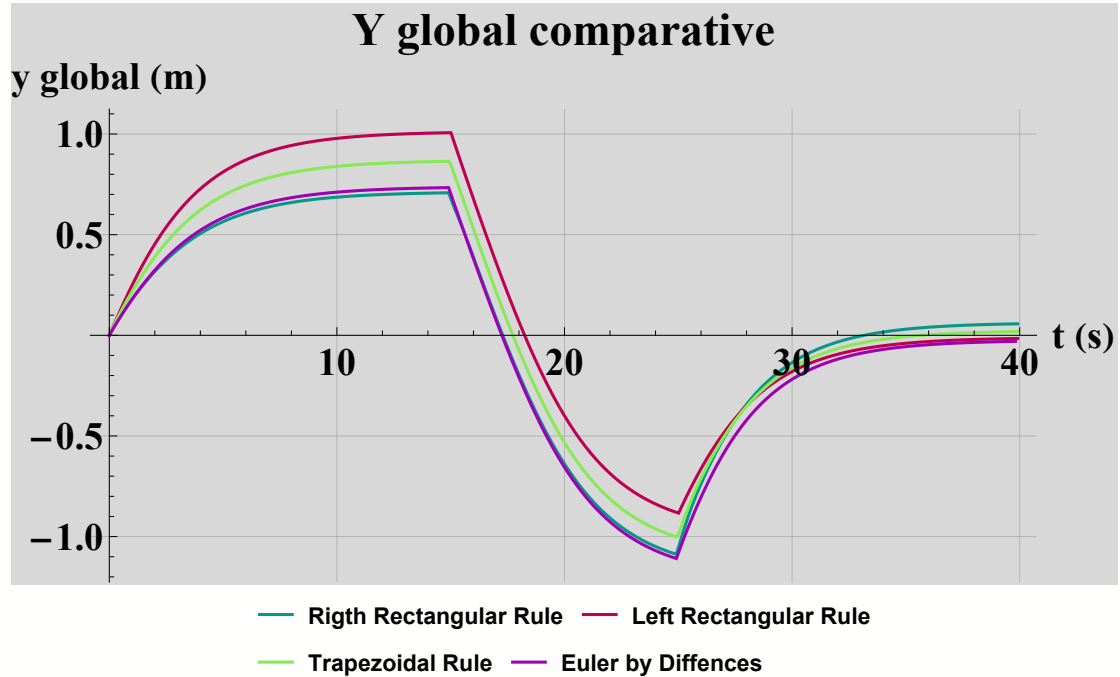
```
Out[320]=
```



In[321]:=

```
grafYglobal = graphs5[{yglobA2, yglobC14, yglobD20, yglobF32},
  "Y global comparative", {"Rigth Rectangular Rule", "Left Rectangular Rule",
    Trapezoidal Rule, "Euler by Diffences"}, {"t", "y global"}, {"s", "m"}]
```

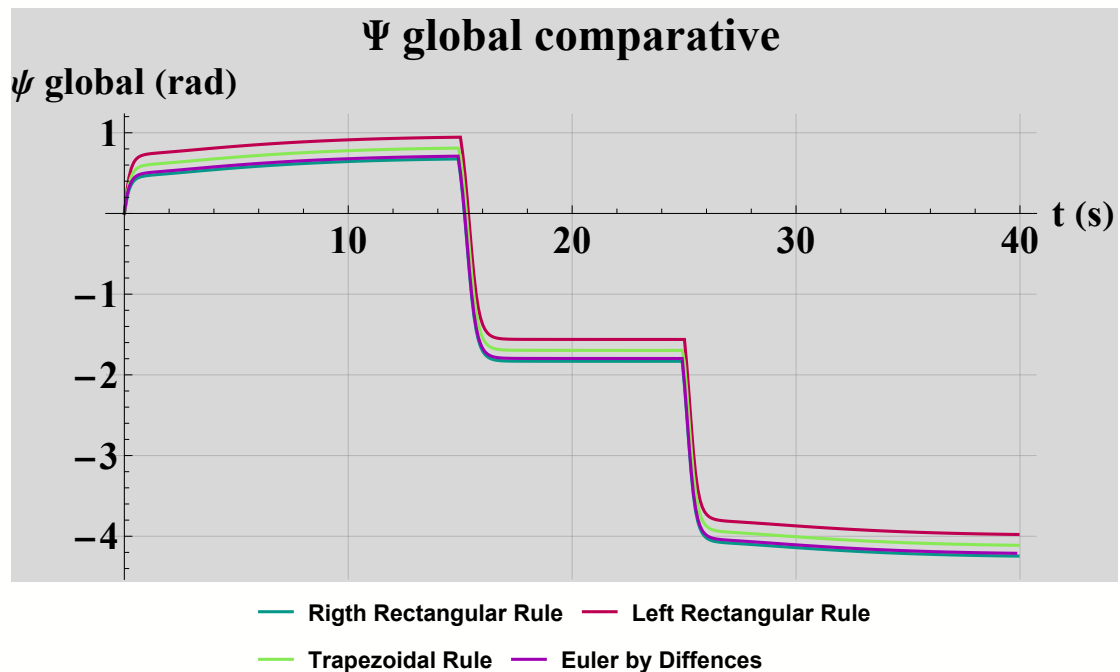
Out[321]=



In[323]:=

```
grafPSIglobal = graphs5[{ψglobA3, ψglobC15, ψglobD21, ψglobF33}, "Ψ global comparative",
  {"Rigth Rectangular Rule", "Left Rectangular Rule", "Trapezoidal Rule", "Euler by Diffences"},
  {"t", "ψ global"}, {"s", "rad"}]
```

Out[323]=



In[324]:=

```

{traj1, traj2, traj3, traj4} = {Transpose[{xglobA1[[A11, 2]], yglobA2[[A11, 2]]}],
                                |transposição |tudo |tudo
                                Transpose[{xglobC13[[A11, 2]], yglobC14[[A11, 2]]}], Transpose[
                                |transposição |tudo |tudo |transposição
                                {xglobD19[[A11, 2]], yglobD20[[A11, 2]]}], Transpose[{xglobF31[[A11, 2]], yglobF32[[A11, 2]]}]]];
                                |tudo |tudo |transposição |tudo |tudo

```

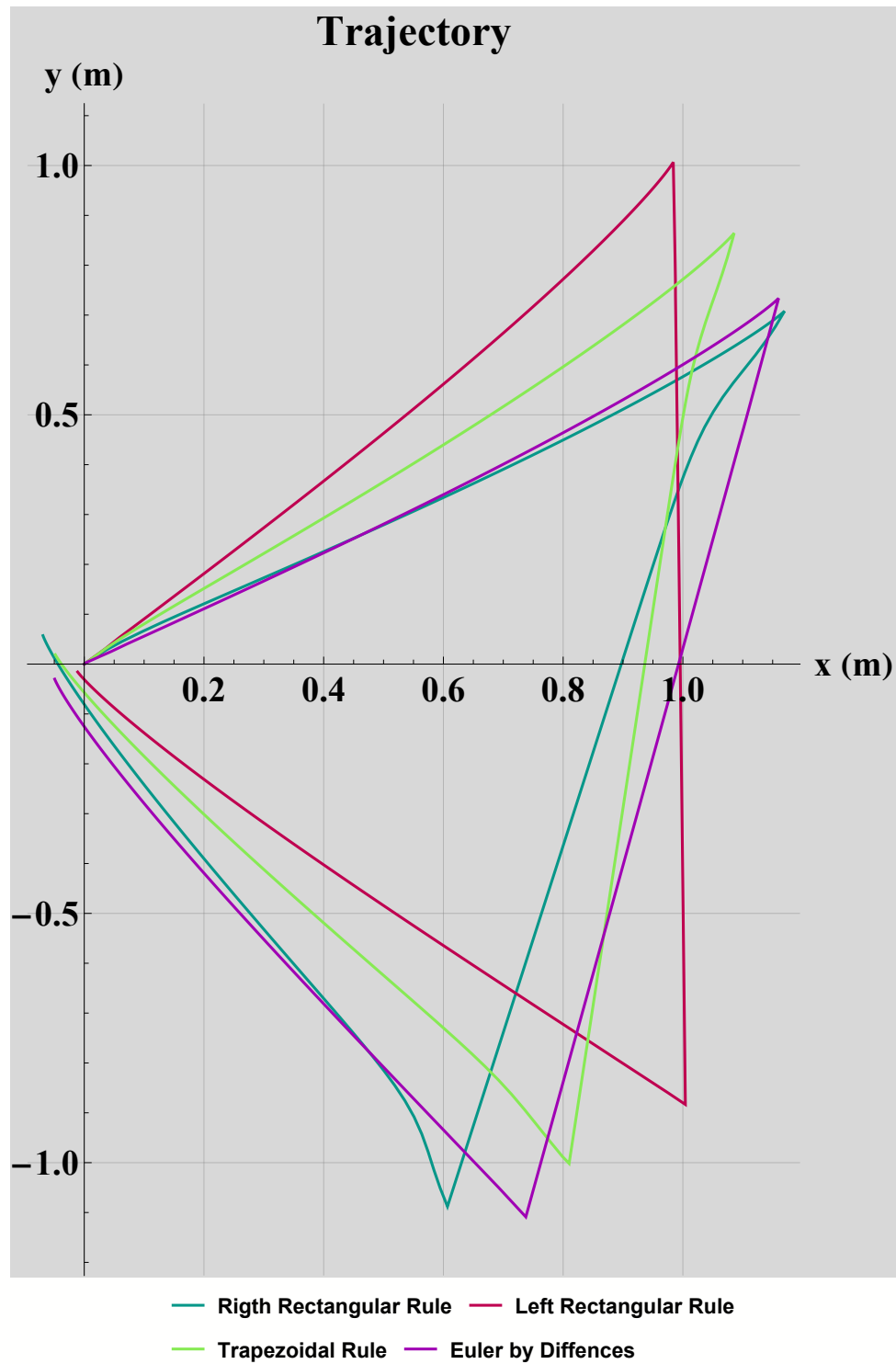
In[325]:=

```

grafTrajectory = graphs4[{traj1, traj2, traj3, traj4},
  "Trajectory", {"Rigth Rectangular Rule", "Left Rectangular Rule",
    Trapezoidal Rule", "Euler by Diffences"}], {"x", "y"}, {"m", "m"}]

```

Out[325]=



Out[331]=

ψ_0 (rad)	xf (m)	yf (m)	ψ_f (rad)	Δx (m)	Δy (m)	$\Delta \psi$ (rad)	Co
0	-0.0689161	0.0570577	-4.24586	-0.0689161	0.0570577	-4.24586	4.5
0	-0.0109789	-0.0158117	-3.97803	-0.0109789	-0.0158117	-3.97803	4.6
0	-0.0483002	0.0188688	-4.11194	-0.0483002	0.0188688	-4.11194	4.6
0	-0.0493439	-0.0301882	-4.21086	-0.0493439	-0.0301882	-4.21086	4.6

columns 7-16 of 16

Out[337]=

Método	N	tf (s)	xf (m)	yf (m)
Right Rectangular Rule	400	39.9121	-0.0689161	0.057
Left Rectangular Rule	400	39.9121	-0.0109789	-0.01
Trapezoidal Rule	400	39.9121	-0.0483002	0.018
Euler by Differences	400	39.8116	-0.0493439	-0.03

Out[340]=

```

"Método"      "N"      "tf (s)"      "xf (m)"      "yf (m)"      "ψf (rad)"
"Right Rectangular Rule"      400      39.9121
      -0.0689160918763494      0.05705770740595478      -4.2458563199999935
"Left Rectangular Rule"      400      39.9121
      -0.010978908723909556      -0.015811665423131508      -3.9780306225000026
"Trapezoidal Rule"      400      39.9121
      -0.04830018373959208      0.018868839100719494      -4.111943471250001
"Euler by Differences"      400      39.8116
      -0.04934386810494021      -0.03018816272349789      -4.210859632499994

```

Out[345]=

Método	tf (s)	xf (m)	yf (m)
Right Rectangular Rule	39.9121	-0.0689161	0.0570577
Left Rectangular Rule	39.9121	-0.0109789	-0.0158117
Trapezoidal Rule	39.9121	-0.0483002	0.0188688
Euler by Differences	39.8116	-0.0493439	-0.0301882